

Written 2026-08-01, after the same three-line bug was written four separate times by the same author across four sessions.

Will: "i feel like we have encountered the formes problem a thousand times please finally fix" and "i want all engines to basically refer to artifacts we update, so that way a dozen different engines dont use a dozen different artifacts? is this the best practice?"

Sharing the artifact was never the problem, and would not have prevented the bug.

Every engine in ABRA already read the *same* table — `MC.mons` , published by the single generated file `data/engine-data.js` . One artifact, one source of truth, exactly the arrangement Will proposed. It broke anyway, because one artifact had **five private doorways**:

file	how it resolved a species name	verdict		
<code>medicham2-browser.js</code>	own <code>pasteKey()</code> — lowercase, spaces→hyphens, linear rescan	worked		
<code>merge_mega_into_engine.js</code>	own <code>byNorm Map</code>	worked		
<code>board.js</code>	<code>MC.mons[norm(x)]</code>	broken — 101 of 308 keys unreachable		
<code>backtest_winrate.js</code>	<code>.map(norm).filter(n => MC.mons[n])</code>	broken — silently deleted every forme		
<code>forced_switch_audit.js</code>	<code>MC.mons[norm(x)]</code>	broken — <code>null</code> for every forme		
<code>type_coverage.js</code>	<code>`MC.mons[key] \</code>	<code>\</code>	<code>MC.mons[norm(key)]`</code>	broken for normalised names

`MC.mons` keys formes with a hyphen (`rotom-wash`); the project's `norm()` strips punctuation. The mismatch cost **8.17% of all observed metagame usage** — Floette-Eternal, Ninetales-Alola, Rotom-Wash, five Hisuian formes — and it cost it *silently*, because a missing key reads as "the engine has never seen this Pokémon", which is a legitimate condition nothing should complain about.

The unit that has to be single-source is the FUNCTION THAT READS the data, not just the data.

Three established ideas apply, and they stack.

`species` is a `string` . `norm(species)` is *also* a `string` . Nothing — not the compiler, not the runtime — stops you handing the second where the first was meant. This is the textbook [primitive obsession](#) smell: using a raw primitive to carry a domain concept that has rules. The standard corrective is a [strongly typed identifier](#) or value object, so that a "display name" and a "table key" are different *types* and cannot be swapped by accident.

We are in plain JavaScript with no compiler, so we cannot get that check for free. What we can do is the next best thing: **make the raw table unreachable and hand out only an accessor**, which is the same idea enforced socially instead of by a type system.

[Poka-yoke design](#) is the Lean principle of building an interface so the incorrect use is difficult or impossible; in software this is just encapsulation. A comment saying "use `mcKey`" is not poka-yoke. A test that fails the build when you don't is.

Concretely: **the accessor takes the RAW name a human or a sheet would write, and does its own normalising.** Callers never construct a key. There is then no wrong string to pass.

An [architecture fitness function](#) sits between a unit test (does this return the right value) and a linter (is this formatted right): it asserts that the *shape* of the system still matches intent — "all database calls still go through the repository layer", "module A still does not import module B". Tools like ArchUnit exist for this in typed languages; in this repo a `grep`-based test does the same job.

This matters because a behavioural test only proves *today's* callers work. The recurring failure here is a **new** caller writing the same three lines again — which no behavioural test can catch.

The operational loop, from [Factory.ai's write-up on using linters to direct agents](#): observe the recurring anti-pattern → codify it as a rule → run it repo-wide to list violations → mass-fix → put the rule on the hot path so it cannot land twice. The result is a codebase that "self-heals: new issues become rules ... and guardrails prevent the same problem from landing twice."

The shared unit is the reader, not the file. `engine/mc_key.js` is the only thing allowed to know how `MC.mons` spells a species.

Enforced by: `tests/test-mc-key.js` — fails on any new file indexing `MC.mons` with a computed key or building its own index over its keys.

Will's own caution, and it is right: "*some callers will need different things from certain artifacts.*" That is a reason for the accessor to take an **argument**, never a reason for a second lookup. The repo already has the good example — `species_sets.js:23`: the bot samples within `cover(sp, 0.80)`, DITTO's gauntlet keeps the entire tail, because "optimising a team against opponents who all run the modal set is optimising against a metagame that does not exist." One accessor, one knob, two callers, opposite needs, no duplication.

Share the FACT. Do not share the POLICY over the fact. And some things must not be shared at all — Will's Tower games are quarantined and never trained on, by design.

`mcKey('Rotom-Wash')`, `mcKey('rotom wash')` and `mcKey('rotomwash')` all answer `rotom-wash`. There is no normalised-but-not-quite string for a caller to get wrong, because callers never hold one.

`magnemite.js` already compared feature names and vector length, and both passed while `allyHit` quietly changed meaning. `engine/feature_fixture.js` hashes each feature's values over frozen boards and checks them at weight load, so a changed *meaning* fails loudly.

Enforced by: `tests/test-feature-antics.js`, which proves the guard catches the real 2026-08-01 defect rather than asserting it in the abstract.

Baseline what exists; fail only on what is **new**. `tests/test-no-silent-failure.js` was adopted this way with 233 pre-existing cases and is still running; a test that demands a 233-file cleanup before it can be turned on gets turned off. `data/mc-key-baseline.json` holds the 16 files still hand-rolling the lookup. **The list may only shrink.**

Two entries there are legitimate and will stay: `medicham2-browser.js` is a browser file with no `require`, and `merge_mega_into_engine.js` *builds* the table, so it cannot ask an index of something that does not exist yet.

This is the rule that would have ended the forme problem in July. Fixing an instance is not finishing; if the same shape has now happened twice, the fix is not done until something *fails* when it happens again.

The repo already had this instinct and applied it unevenly — `test-drop-guard.js` asserts `B.featuresFor()` appears exactly once, precisely so a second call site cannot reappear. That is the pattern. It was simply never applied to species lookup.

The semantics guard did **not** catch the forme bug, and that is worth stating rather than glossing: every species in its fixture was hyphen-free, so the entire forme code path lay outside the boards it checked. Coverage is not incidental to a fitness function, it is the function. `test-mc-key.js` therefore asserts every key in the live table resolves, rather than spot-checking a few.

- `engine/mc_key.js` — the one resolver, node + browser, index built from the artifact's own keys so nothing is typed and a change to the generator's naming is picked up with no edit.
- `board.js` **delegates to it** rather than keeping a fourth private copy. Verified behaviour-identical by the semantics hashes not moving.
- **Fixed:** `backtest_winrate.js` (was deleting every forme team from the MEDICHAM validation set), `forced_switch_audit.js`, `type_coverage.js`.
- `tests/test-mc-key.js` — the fitness function, ratcheted against `data/mc-key-baseline.json`.

§5 below predicted that the next application of R1 would be another artifact. It was, one day later, and it was found by looking for the SHAPE rather than by hitting the bug: seven files replaying the store, each writing its own

```
sheet[base(m.species)] = { side, moves: ... }; // ...and never reading `side` again
```

`engine/fit_policy.js`, `engine/fit_joint.js`, `engine/branch_recall.js`, `engine/feature_coverage.js`, `engine/ko_calibration.js`, `engine/surprise.js`, `tests/test-degradation-budgets.js`.

Species Clause limits one of each Pokémon **per player**, not per battle, so a species-only key collapses the two team sheets in a mirror. Measured by `engine/redirect_audit.js` over 7,454 open-sheet games:

games with a species on BOTH sheets	58.63%
slots scored against the OTHER side's four moves	8.02%
...of those, matched anyway and fitted against the wrong choice set	62.16%

The silent half is the damaging one. A slot that fails to match gets counted and dropped; a slot that matches against the opponent's moveset is a **wrong denominator** in a conditional logit — the same defect `board.js`'s choice-lock note describes — and nothing counted it at all.

- `engine/click_match.js` — one reader for "whose moveset is this, and which candidate did they press". Side-keyed, forme-folded through the dex's own `baseSpecies`, and it resolves a recorded target back through the turn's own switches.
- `engine/joint_rows.js` — the pair-decision replay loop, extracted from `fit_joint.js` so that asking a question about the pair fit does not mean writing a fourth copy of it. Verified against the shipped artifact's own tally: 86,242 turns seen, 66,236 kept, 19,995 unmatched, 11 ambiguous.
- `tests/test-click-match.js` — the fitness function. Every behavioural assertion is built from a measured defect and *also* asserts the old lookup fails, so it cannot pass both before and after.

Two more defects fell out of the same measurement, both of the same family — a lookup that answers "never seen it" when the truth is "you asked wrongly":

- **A human targets a SLOT; the store records a SPECIES.** Switches resolve before moves, so the protocol writes down the mon that *arrived* while the human was choosing against the one that *left*. 44.37% of every failed match, the single largest cause.

- **In-battle forme changes have no sheet entry.** floette 3,627, then aegislashblade , palafinhero , mimikyubusted , morpekohangry . 19.65% of failures. The forme problem again, in a third table.

Net, on the same replay scored twice: usable joint turns **76.80% → 94.52%**, drop **23.18% → 5.47%**.

And it corrected a diagnosis that three documents were quoting. MODELS.md , DEFENSE.md and fit_policy.js 's own caveat all said the unmatched clicks were "mostly redirection". Redirection is **1.60%** of them. The protocol never records a move's *chosen* target, only its resolved one, so redirection cannot make a click unmatchable — it can only make the label wrong, which it does to 1.55% of clicks. **A cause nobody measured had been repeated until it read as established.**

- **R1 is enforced by regex, not by a type system.** A caller determined to get around it can. The regex is deliberately over-broad and then baselined, because a clever regex that decides which lookups are "fine" is how the next one gets through.
- **This document does not fix the 16 baselined files.** Most are tests indexing with keys they themselves just produced, which is harmless. They are recorded so the number can only go down.
- MC.moves **still has not been swept.** It happens to be collision-free and hyphen-free today, so it has no equivalent bug — but nothing checks that it stays that way. tests/test-artifact-keys.js is the general detector; it flags a table only once the keys are actually unsafe, which is later than finding it by shape.
- **Redirection mislabels 1.55% of clicks and this does not fix it.** The chosen target was never written down, by anyone, at any point — it is not recoverable from the store, only from a re-ingest that does not exist. Stated rather than papered over.